

LuxLLM-Agent: An Explainable LLM-Assisted Agent and Replay Viewer for Lux AI Season 3

First Author

Affiliation / Address line 1
email@domain

Second Author

Affiliation / Address line 1
email@domain

Abstract

We present LuxLLM-Agent, an LLM-assisted agent and replay viewer for Lux AI Season 3, designed around system-level, replay-grounded explainability. The system combines a local game-playing agent, rule-based fallback policies, structured decision traces, and an S1-style isometric battle replay viewer for inspecting agent behaviour. Rather than claiming to explain the internal neural computation of the LLM, the system exposes decision sources, fallback status, risk-filter changes, unit actions, match-score context, and replay-frame alignment. We evaluate the system through controlled matches against a rule-based baseline, trace-based replay inspection, replay-frame verification, and a lightweight scalability simulation. The controlled match results are treated as preliminary descriptive evidence rather than statistically conclusive model rankings, and the artifact package includes the viewer, replay data, paper figures, reports, source code, and demo media.

1 Introduction

Game AI competitions provide a useful testbed for studying decision making, planning, and multi-agent behaviour under partial observability and limited runtime. The Lux AI Challenge provides a competitive game-AI environment for studying multi-variable optimisation, resource gathering, allocation, and opponent-aware policy design (Lux AI Challenge, 2024a,b). Lux AI Season 3 further frames this setting as a two-player competition with repeated match sequences and resource-oriented decision making (Lux AI Challenge, 2024c). Lux AI Season 3 is especially challenging because an agent must explore an initially uncertain map, reason about relics and energy, control multiple units, and react to an opponent over many steps.

Large language models (LLMs) can provide flexible high-level reasoning, and recent LLM-agent systems have explored reasoning-and-acting loops,

verbal self-feedback, open-ended embodied agents, and generative interactive agents (Yao et al., 2023; Shinn et al., 2023; Wang et al., 2023; Park et al., 2023). However, directly using LLMs inside a real-time game loop introduces latency, instability, and reproducibility concerns.

This paper presents LuxLLM-Agent, a system demonstration that focuses on making LLM-assisted game agents inspectable, traceable, and reproducible. The system uses an LLM-assisted strategy layer with rule-based fallback, records structured decision traces, and provides a replay viewer that visualises both the battle state and the logged decision evidence. The target users are researchers and students interested in LLM agents, game AI, agent evaluation, and replay-grounded analysis of agent behaviour.

In this paper, we use explainability in a system-level and replay-grounded sense. The system does not attempt to explain the internal neural computation of the LLM. Instead, it exposes the decision pathway used by the agent system: the decision source, active strategy, fallback status, risk-filter changes, unit-level actions, match-score context, and replay-frame alignment. This allows users to inspect how the agent behaved over time and to connect observed replay events with the logged decision evidence. The central motivation is that the outcome of a game match is not enough to understand whether an LLM-assisted agent is behaving reliably. A final score does not show whether an LLM decision was fresh or stale, whether a rule fallback was used, whether a target was changed by a risk filter, or whether the agent followed a coherent strategy over time. For this reason, LuxLLM-Agent treats replay visualisation as part of the agent evaluation pipeline rather than as a separate cosmetic interface. The viewer is designed to make the agent’s logged decision pathway inspectable alongside the replay, so that users can connect state changes, score changes, and fallback events with

the corresponding match context.

Our contributions are:

- an LLM-assisted Lux AI Season 3 agent with cached strategy decisions and rule-based fallback for runtime stability;
- a trace-based logging pipeline that records agent strategies, decision sources, fallback events, risk-filter changes, unit actions, and match-level outcomes;
- an S1-style isometric battle replay viewer that turns Lux S3 logs into a replay-grounded inspection interface for demonstration, debugging, and paper figures;
- a controlled evaluation, trace-availability analysis, and lightweight scalability simulation, with the current match results reported as preliminary descriptive evidence rather than statistically conclusive model rankings.

2 System Overview

LuxLLM-Agent consists of four main components: an LLM-assisted agent, a rule-based fallback policy, a structured trace logging and evaluation pipeline, and a replay-driven visualisation interface. During a match, the agent observes the current game state and converts it into a compact decision context. When the LLM is available within the runtime budget, it produces a high-level strategy such as exploration, energy collection, relic movement, or scoring. If the LLM call is unavailable, stale, or unsafe, the system falls back to deterministic rule policies. This design treats the LLM as a sparse strategic planner rather than as a controller for every unit-level action.

The system is designed around system-level explainability through traceability. Each important decision is logged with the current strategy, decision source, fallback status, unit action, and risk-filter information. These traces do not claim to expose the internal neural reasoning process of the LLM. Instead, they expose the decision pathway used by the implemented agent system. The logs are later converted into replay frames and evaluation summaries. The replay viewer then reconstructs a visual battle timeline from these frames, allowing users to inspect how the agent behaved over time rather than only seeing the final score.

The end-to-end data flow is intentionally simple and reproducible. A local Lux AI Season 3 match

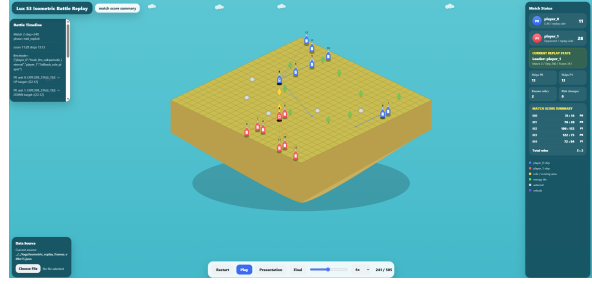


Figure 1: S1-style isometric battle replay viewer for Lux AI Season 3. The viewer loads merged replay frames and shows both players, unit positions, score, match state, timeline, and per-match score summary.

produces console logs and structured frame logs. A replay-frame builder merges the relevant per-step evidence into a compact JSON representation. The viewer loads this JSON file and renders the game state as a floating-island battle map. Finally, screenshots, verification reports, and artifact indices are generated from the same replay package so that the paper, demo video, and repository all refer to the same evidence source. This shared evidence source reduces the risk that the paper figures, video demonstration, and downloadable artifact describe different versions of the system.

2.1 Implementation

The prototype is implemented in Python and JavaScript. The Lux AI Season 3 agent runs locally through the competition environment, while LLM decisions are produced through a local Ollama backend using Qwen2.5 models (Yang et al., 2024). The implementation records console logs, structured JSONL frame logs, evaluation JSON files, replay frames, screenshots, and Markdown summaries. The viewer is implemented as a standalone HTML interface that can be opened through a local HTTP server and used for live demonstration or video recording.

The current demonstration package includes a stable S3 isometric battle replay viewer, merged replay JSON, four paper-ready screenshots, a demo video, controlled evaluation summaries, minimal source code, and scalability reports. The system does not require cloud LLM APIs for the local demonstration. This makes the artifact suitable for reviewer inspection because the replay viewer can be launched locally without rerunning the full game environment or making external model calls.

2.2 Design Choices

The system separates high-level strategy generation from low-level action execution. This choice is important because a full Lux AI Season 3 match requires many small unit-level decisions, while the LLM is better suited to sparse strategic reasoning. The rule-based fallback layer ensures that the agent can continue acting even when the LLM is unavailable, slow, or produces an unusable response. This makes the system more robust than a design that depends on an LLM for every action.

The viewer is designed in the style of the Season 1 floating-island replay interface so that the Season 3 demonstration remains visually consistent with earlier Lux-style visualisations. At the same time, the interface adds Season 3-specific evidence such as match score summaries, final result overlays, known relic counts, risk-change counts, and replay-frame coverage. The interface is also motivated by human-AI interaction principles: users should be able to understand system status, inspect behaviour, and recover context when automated decisions are uncertain or difficult to interpret (Amer-shi et al., 2019). The result is a viewer that is both presentation-friendly and useful for debugging agent behaviour.

3 Demonstration Walkthrough

A typical demonstration begins by opening the S3 isometric battle replay viewer. The viewer loads the prepared replay-frame JSON and renders the match as an S1-style floating-island battle map. The user can scrub through the match timeline, observe both player_0 and player_1 units, inspect the current score, and view the per-match score summary. The left panel presents a compact battle timeline, while the right panel shows match status, ship counts, known relics, risk changes, and match-level scores.

The demonstration is organised around a short reviewer-facing workflow. First, the viewer loads the replay JSON and reconstructs the initial battle state. Second, the user plays or scrubs through the timeline to observe how both players move across the map. Third, the user inspects replay-aligned evidence such as score changes, known relics, ship counts, and risk-filter changes. Fourth, the user switches to Presentation Mode, which reduces auxiliary panels and makes the map easier to record in a short screencast. Finally, the user opens the final result overlay to view the winner, final score, replay coverage, risk-filter statistics, total match

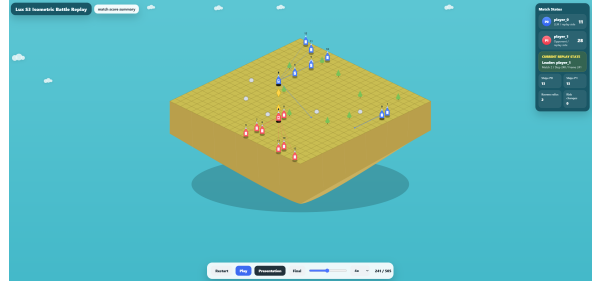


Figure 2: Recording-friendly Presentation Mode of the S3 battle replay viewer. Auxiliary panels are reduced so that the central floating-island battle map becomes the visual focus for demo-video recording.

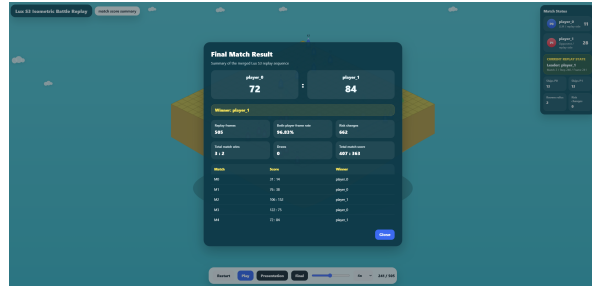


Figure 3: Final result overlay in the S3 replay viewer. The overlay summarises the final winner, final score, replay-frame coverage, total risk-filter changes, total match wins, and total score.

wins, and total score.

The walkthrough is intended to support replay-grounded inspection rather than only visual playback. A standard replay can show where units moved, but it does not expose whether a decision came from an LLM strategy, a deterministic fallback, or a risk-filtered target adjustment. LuxLLM-Agent addresses this by aligning the visual replay with structured trace evidence. This makes it possible to inspect not only what happened in the match, but also which logged decision pathway was active when the behaviour occurred.

This workflow is designed for both research inspection and demonstration. A researcher can use the viewer to locate suspicious behaviour, fallback-heavy intervals, sudden score changes, or risk-filter events. A supervisor or reviewer can use the same viewer to understand the system without reading raw logs. The paper screenshots and screencast are generated from this interface, so the visual evidence in the paper is directly connected to the downloadable artifact.

This workflow is designed for both research inspection and demonstration. A researcher can use the viewer to locate suspicious behaviour, fallback-

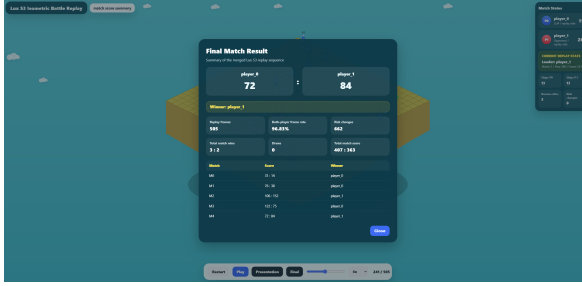


Figure 4: Final result overlay in the S3 replay viewer. The overlay summarises the final winner, final score, replay-frame coverage, total risk-filter changes, total match wins, and total score.

heavy intervals, or sudden score changes. A supervisor or reviewer can use the same viewer to understand the system without reading raw logs. The paper screenshots and screencast are generated from this interface, so the visual evidence in the paper is directly connected to the downloadable artifact.

4 Evaluation

We evaluate the system from four perspectives: controlled match outcome, trace-based explainability support, replay evidence quality, and architecture-level scalability. The goal of the evaluation is not to claim that the system has solved Lux AI Season 3, nor to claim that the internal reasoning of the LLM is fully explained. Instead, the evaluation checks whether the system can run controlled matches, record useful decision traces, align those traces with replay frames, and provide a downloadable artifact that supports post-hoc inspection.

4.1 Controlled Match Results

In the controlled match setting, 57 matches were run across three configurations: rule-based baseline versus rule-based baseline, Qwen2.5 1.5B versus rule baseline, and Qwen2.5 7B versus rule baseline. The results are reported as descriptive statistics from limited controlled runs. They should not be interpreted as a definitive ranking of model capability.

In this limited setting, the Qwen2.5 1.5B configuration shows a slightly higher observed win rate than the 7B configuration. However, the difference is only one additional win over 19 matches and should not be interpreted as statistically conclusive. We therefore treat this result as preliminary evidence that smaller local models may provide a favourable runtime–performance trade-off under the

Configuration	Matches	Wins	Win Rate
Rule vs Rule	19	9	47.37%
Qwen2.5 1.5B vs Rule	19	14	73.68%
Qwen2.5 7B vs Rule	19	13	68.42%

Table 1: Controlled win-rate comparison across rule and LLM-assisted configurations. The results are descriptive statistics from a limited controlled setting.

Capability	Raw Logs	Replay Only	Ours
Decision source visible	Yes	No	Yes
Fallback status visible	Yes	No	Yes
Risk-filter changes visible	Yes	No	Yes
Replay-frame alignment	No	Yes	Yes
Match score summary	No	Partial	Yes
Final result overlay	No	Partial	Yes

Table 2: System-level explainability capability comparison. LuxLLM-Agent combines structured decision traces with replay-grounded visual inspection.

current system constraints, rather than as proof that Qwen2.5 1.5B is generally stronger than Qwen2.5 7B.

Given the small sample size, these results should be interpreted cautiously. A larger evaluation with more random seeds, matched match conditions, formal significance testing, and component-level ablations is required before making stronger claims about model-size effects. In particular, future experiments should compare model configurations under the same random seeds and report not only win rate, but also average score, runtime, fallback count, LLM-call count, stale-decision count, and risk-filter changes.

4.2 Trace-Based Explainability Evaluation

We evaluate explainability as trace availability and replay-grounded inspection support. The goal is not to claim access to the internal reasoning process of the LLM, but to expose the decision pathway used by the implemented agent system at each replay step. Compared with raw console logs or a replay-only viewer, LuxLLM-Agent combines structured decision traces with visual replay context, making it easier to inspect when the system used an LLM decision, when it fell back to deterministic rules, and when risk filtering changed a unit target.

The verified replay package contains 505 replay frames, with both players present in 489 frames, corresponding to 96.83% both-player replay coverage. The replay records 662 risk-filter changes and aligns these events with match-level replay context.

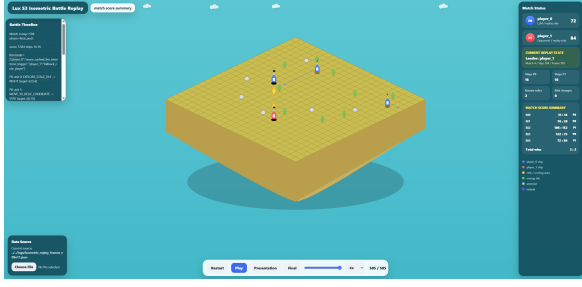


Figure 5: Per-match score summary in the S3 replay viewer. The panel displays individual match outcomes and total match wins over the replay sequence.

These statistics evaluate explanation availability and replay alignment, rather than human-rated explanation quality. We do not claim that the traces are causally faithful explanations of all internal LLM computations. Instead, they provide system-level evidence about which implemented decision pathway was active during replayed behaviour.

4.3 Replay Evidence Verification

The replay viewer was verified using the prepared S3 replay package. The package contains 505 replay frames and supports inspection of both players, match scores, risk-filter changes, and final replay outcomes. Across the representative replay sequence, the total match wins are 3:2 and the total score is 407:363. The final replay frame reports player_1 as the final-match winner with a final score of 72:84.

This replay verification matters because the paper figures, demo video, and downloadable artifact are generated from the same replay package. As a result, the visual evidence shown in the paper is aligned with the local viewer and the JSON artifact included in the repository. This reduces the risk of presenting screenshots or videos that cannot be reproduced from the downloadable package.

4.4 Scalability Simulation

To evaluate scalability without overstating full-game throughput, we implemented a synthetic lightweight-worker simulation. The runner instantiates 10, 100, 500, and 1000 worker agents, each executing cached policy templates for 100 decision steps. The simulation does not launch full Lux AI Season 3 matches and does not invoke an LLM for every worker. Instead, five synthetic LLM calls represent sparse strategy-template generation. The 1000-worker setting completes 100,000 lightweight decisions in 0.807 seconds, while the LLM-

Agents	Decisions	LLM Calls	Runtime (s)	Memory (MB)
10	1,000	5	0.008	0.346
100	10,000	5	0.079	1.214
500	50,000	5	0.409	5.072
1000	100,000	5	0.807	9.894

Table 3: Architecture-level scalability simulation using lightweight worker agents. The runner uses cached policy templates and does not invoke an LLM once per worker.

Capability	Raw Logs	Replay Only	Ours
Decision source visible	Yes	No	Yes
Fallback status visible	Yes	No	Yes
Risk-filter changes visible	Yes	No	Yes
Replay-frame alignment	No	Yes	Yes
Match score summary	No	Partial	Yes
Final result overlay	No	Partial	Yes

Table 4: System-level explainability capability comparison. LuxLLM-Agent combines structured decision traces with replay-grounded visual inspection.

call count remains fixed at 5. This result supports the intended scaling mechanism: expensive strategic reasoning is amortised across many low-cost worker decisions.

This scalability result should be interpreted carefully. It is an architecture-level lightweight-worker simulation rather than a full Lux benchmark. The reported runtime measures lightweight policy execution and synthetic worker-state updates, not game-engine stepping, rendering, or 1000 complete Lux matches. The purpose of this experiment is to test the system design claim that sparse LLM-generated policy templates can be reused across many worker agents, rather than to claim that the full Lux environment has been run with 1000 complete LLM-controlled players.

4.5 Trace-Based Explainability Evaluation

We evaluate explainability as trace availability and replay-grounded inspection support. The goal is not to claim access to the internal reasoning process of the LLM, but to expose the decision pathway used by the agent system at each replay step. Compared with raw console logs or a replay-only viewer, LuxLLM-Agent combines structured decision traces with visual replay context, making it easier to inspect when the system used an LLM decision, when it fell back to deterministic rules, and when risk filtering changed a unit target.

The verified replay package contains 505 replay frames, with both players present in 489 frames, corresponding to 96.83% both-player replay cover-

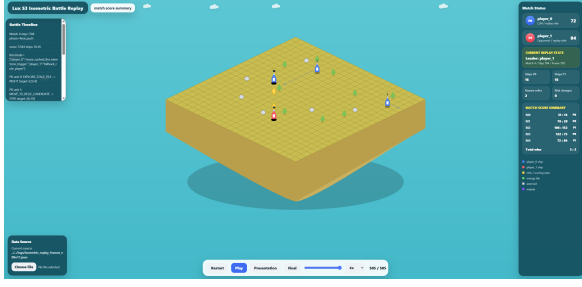


Figure 6: Per-match score summary in the S3 replay viewer. The panel displays individual match outcomes and total match wins over the replay sequence.

age. The replay records 662 risk-filter changes and aligns these events with match-level replay context. These statistics evaluate explanation availability and replay alignment, rather than human-rated explanation quality. We do not claim that the traces are causally faithful explanations of all internal LLM computations. Instead, they provide system-level evidence about which implemented decision pathway was active during replayed behaviour.

4.6 Replay Evidence Verification

The replay viewer was verified using the prepared S3 replay package. The package contains 505 replay frames and supports inspection of both players, match scores, risk-filter changes, and final replay outcomes. Across the representative replay sequence, the total match wins are 3:2 and the total score is 407:363. The final replay frame reports player_1 as the final-match winner with a final score of 72:84.

This replay verification matters because the paper figures, demo video, and downloadable artifact are generated from the same replay package. As a result, the visual evidence shown in the paper is aligned with the local viewer and the JSON artifact included in the repository. This reduces the risk of presenting screenshots or videos that cannot be reproduced from the downloadable package.

4.7 Scalability Simulation

To evaluate scalability without overstating full-game throughput, we implemented a synthetic lightweight-worker simulation. The runner instantiates 10, 100, 500, and 1000 worker agents, each executing cached policy templates for 100 decision steps. The simulation does not launch full Lux AI Season 3 matches and does not invoke an LLM for every worker. Instead, five synthetic LLM calls represent sparse strategy-template generation.

Agents	Decisions	LLM Calls	Runtime (s)	Memory (MB)
10	1,000	5	0.008	0.346
100	10,000	5	0.079	1.214
500	50,000	5	0.409	5.072
1000	100,000	5	0.807	9.894

Table 5: Architecture-level scalability simulation using lightweight worker agents. The runner uses cached policy templates and does not invoke an LLM once per worker.

The 1000-worker setting completes 100,000 lightweight decisions in 0.807 seconds, while the LLM-call count remains fixed at 5. This result supports the intended scaling mechanism: expensive strategic reasoning is amortised across many low-cost worker decisions.

This scalability result should be interpreted carefully. It is an architecture-level lightweight-worker simulation rather than a full Lux benchmark. The reported runtime measures lightweight policy execution and synthetic worker-state updates, not game-engine stepping, rendering, or 1000 complete Lux matches. The purpose of this experiment is to test the system design claim that sparse LLM-generated policy templates can be reused across many worker agents, rather than to claim that the full Lux environment has been run with 1000 complete LLM-controlled players.

4.8 Ablation and Statistical Testing Plan

The current evaluation is sufficient for a system demonstration, but it is not yet a full empirical study of every system component. Future work should include ablation studies that separate the effects of model size, rule-based fallback, risk filtering, stale-decision guards, and replay logging. Planned configurations include rule-only policies, LLM-assisted policies without risk filtering, LLM-assisted policies with fallback disabled, and full-system variants under matched random seeds.

Future experiments should also increase the number of matches per condition and apply formal statistical tests when comparing win rates across model configurations. For example, Fisher’s exact test or two-proportion tests could be used to compare win-rate differences, while runtime, fallback count, LLM-call count, and risk-filter changes could be reported as supporting operational metrics. This would make it possible to distinguish robust system effects from run-to-run variance.

5 Comparison with Existing Systems

Prior game-AI systems have demonstrated that complex games can serve as demanding environments for planning, learning, and multi-agent coordination. Examples such as AlphaStar, OpenAI Five, and MuZero show the importance of scalable decision systems, planning, and large-scale evaluation in game domains (Vinyals et al., 2019; Berner et al., 2019; Schrittwieser et al., 2020). LuxLLM-Agent is different in scope: it is not a large-scale reinforcement-learning system, but an inspectable LLM-assisted agent and replay-analysis pipeline designed to expose strategy decisions, fallback behaviour, risk-filter events, and replay evidence.

Official Lux replay tools are useful for observing game outcomes, but they are not designed to explain why an LLM-assisted agent selected a strategy, when fallback was triggered, or how risk filtering changed unit targets. Standard console logs provide decision evidence, but they are difficult to interpret visually and do not provide a replay-like user experience. Conversely, a replay-only viewer can show unit movement and scores, but it usually does not expose the decision source or fallback pathway behind a particular action.

LuxLLM-Agent fills this gap by combining agent decision traces, match evaluation, and replay visualisation. The system is not only a game-playing agent and not only a viewer. It is a demonstration pipeline that connects LLM strategy decisions, rule fallback, risk-filter events, match outcomes, and visual replay evidence. This makes it suitable for analysing LLM-assisted game agents in a way that is more inspectable than raw logs and more decision-aware than a standard replay.

Compared with a pure game-playing submission, the system emphasises reproducible inspection. Compared with a pure visual replay tool, it preserves decision evidence from the agent pipeline. Compared with a per-agent LLM controller, it uses cached strategy templates and rule execution so that the LLM plays a sparse strategic role instead of controlling every unit action. This positioning is important for a system demonstration paper because the contribution is not only a game result, but also an inspectable artifact for studying LLM-assisted game-agent behaviour.

6 Limitations and Future Work

LuxLLM-Agent operationalises explainability as system-level, replay-grounded traceability. It ex-

poses decision sources, fallback status, risk-filter changes, unit actions, match scores, and replay-aligned outcomes, but it does not measure neural-mechanistic explanation faithfulness inside the LLM. Future work should evaluate whether these traces improve debugging speed, user trust calibration, and failure-mode identification.

The current controlled evaluation is limited in scale. The Qwen2.5 1.5B configuration wins one more match than the 7B configuration over 19 matches, so this difference should be treated as a preliminary observation rather than a statistically significant model-size effect. Larger evaluations should use more random seeds, matched match conditions, Fisher’s exact test or two-proportion tests, and component-level ablations.

Future ablations should separate the effects of model size, rule-based fallback, risk filtering, stale-decision guards, and replay logging. The current experiments were conducted on a single laptop-class machine with an NVIDIA RTX 4060 Laptop GPU and local Ollama models; stronger institutional compute resources would allow more repetitions, larger model variants, and more reliable statistical testing.

7 Licensing and Availability

The project is released as a downloadable research prototype under the MIT License. For review and submission, the artifact package is available through the anonymous repository link:

<https://anonymous.4open.science/r/luxllm-agent-demo-CED3>

The package includes the S3 isometric battle replay viewer, merged replay JSON, paper figures, a replay-preview GIF, a screencast video, controlled-evaluation summaries, scalability reports, and a minimal source-code package. The viewer can be run locally with Python’s built-in HTTP server and inspected without rerunning the full Lux AI Season 3 environment or making cloud LLM API calls.

8 Conclusion

LuxLLM-Agent demonstrates an explainable LLM-assisted agent pipeline for Lux AI Season 3. The system combines local LLM strategy generation, rule-based fallback, structured decision logging, replay-driven visualisation, controlled evaluation, and lightweight scalability evidence. The results suggest that smaller local LLMs can be effective

under runtime constraints, and that sparse LLM-generated policy templates can be reused across many lightweight worker agents. The demonstration is intended to help researchers and students inspect, reproduce, and discuss LLM-assisted game-agent behaviour.

Ethics Statement

This system is developed for research and educational use in a game AI environment. It does not use personal data and does not make decisions about real users. The main risks are overclaiming system capability and misinterpreting synthetic scalability results as full-game multi-agent performance. We mitigate these risks by clearly separating controlled match evaluation, replay-viewer evidence, and architecture-level scalability simulation. The system should be presented as a research prototype rather than a deployed autonomous decision system.

Acknowledgements

The author thanks the project supervisor and the Lux AI Challenge community for providing the competition environment and motivation for this work.

References

- Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fournery, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. 2019. [Guidelines for human-ai interaction](#). In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pages 1–13.
- Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemyslaw Debiak, Christy Denison, David Farhi, Quirin Fischer, Shariq Hashme, Chris Hesse, and 1 others. 2019. [Dota 2 with large scale deep reinforcement learning](#). *arXiv preprint arXiv:1912.06680*.
- Lux AI Challenge. 2024a. Lux ai challenge. <https://www.lux-ai.org/>. Accessed 2026-06-08.
- Lux AI Challenge. 2024b. Lux ai challenge season 3 design repository. <https://github.com/Lux-AI-Challenge/Lux-Design-S3>. Accessed 2026-06-08.
- Lux AI Challenge. 2024c. Neurips 2024 - lux ai season 3. <https://www.kaggle.com/competitions/lux-ai-season-3>. Accessed 2026-06-08.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. 2023. [Generative agents: Interactive simulators of human behavior](#). In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*.
- Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. 2020. [Mastering atari, go, chess and shogi by planning with a learned model](#). *Nature*, 588:604–609.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. [Reflexion: Language agents with verbal reinforcement learning](#). *Preprint*, arXiv:2303.11366.
- Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michael Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, and 1 others. 2019. [Grandmaster level in starcraft ii using multi-agent reinforcement learning](#). *Nature*, 575:350–354.
- Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023. [Voyager: An open-ended embodied agent with large language models](#). *Preprint*, arXiv:2305.16291.
- An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, and 1 others. 2024. [Qwen2.5 technical report](#). *Preprint*, arXiv:2412.15115.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. [React: Synergizing reasoning and acting in language models](#). In *International Conference on Learning Representations*.

A Additional Details

The supplementary artifact contains replay JSON files, screenshots, a replay-preview GIF, a screen-cast video, source code, and Markdown reports. The verified replay package contains 505 frames, 96.83% both-player frame coverage, and 662 risk-filter changes.